

Interlude: Move, Copy, and Clone

Opening Question

Why does assigning one value to another name sometimes invalidate the first name, sometimes preserve it automatically, and sometimes require an explicit method call?

1. Assignment does not have one universal meaning

Consider two assignments that look identical in shape and behave nothing alike.

```
let a = 5;  
let b = a;  
println!("{a}"); // valid
```

```
let a = String::from("signal");  
let b = a;  
// println!("{a}"); // invalid
```

In the first case, writing `a` after the assignment is unremarkable. In the second, the compiler refuses the same syntactic move. Nothing about the *syntax* `let b = a;` changed between the two examples. What changed is what the value on the right-hand side actually is: an interchangeable quantity in the first case, and a handle to an external allocation in the second. Before naming the mechanisms, it is worth sitting with the phenomenon itself: identical code produces opposite consequences depending on what kind of value is being assigned.

2. Move transfers authority

Chapter 5 already established that ownership is unique: exactly one path in the program has authority over a resource-owning value at any moment. The second example above is what that uniqueness looks like at the point of assignment. Writing `let b = a;` does not duplicate the `String`'s heap allocation. It transfers the authority that `a` held to `b`, and `a` is left holding nothing usable. This is not a special rule bolted onto assignment; it is the ordinary consequence of Chapter 5's reachability set $R(v)$ changing hands. This interlude does not re-derive that claim. It starts from it, in order to ask a narrower question: which values are exempt from it, and why.

3. Copy preserves the source implicitly

Simple scalar types — integers, floats, `bool`, `char` — and small aggregates built entirely from such types do not lose their source on assignment. `let b = a;` produces a second, independent `i32` with the same bit pattern as the first. Both names remain valid because both now refer to fully independent values. Nothing is shared between them, so nothing about ownership's uniqueness constraint is violated by having two of them.

The exemption is granted by the `Copy` trait, and it is granted narrowly. A type may implement `Copy` only if bitwise duplication is a completely adequate substitute for genuine reconstruction — only if the bits contain no reference to anything the type does not itself, fully, contain.

4. Why `String` is not `Copy`

A `String` is not large; a `String` that happened to be one character long is still not `Copy`. The disqualifying property has nothing to do with size. A `String`'s in-memory representation is a pointer, a length, and a capacity — a handle describing an allocation the `String` does not itself physically contain. Bitwise duplication of that handle would produce two independent variables that both believe they own the same heap allocation. Nothing would prevent both from later freeing it, or one from mutating memory the other still expects to be valid. The bits encode authority over something external, and authority, by Chapter 5's account, cannot be silently duplicated without breaking uniqueness. This is precisely why the compiler moves `String` instead of copying them: moving is the operation that preserves uniqueness when duplication cannot.

5. Clone requests reconstruction explicitly

When a genuine second value is wanted — not a second name for the same allocation, but an independent allocation with equal contents — Rust requires that the request be made in the syntax itself.

```
let a = String::from("signal");
let b = a.clone();
println!("{a} and {b}");
```

Both names remain valid here because `.clone()` performed real work: it allocated a second buffer and copied the bytes into it. `a` and `b` now have equal contents but separate storage, separate addresses, and from this point forward, separate futures. Changing one will never be observed through the other. Equality of content is not identity of storage, and `.clone()` is the point in the program where that distinction is made visible rather than assumed.

6. Copy and Clone are related but not interchangeable

Every `Copy` type is also `Clone` — if a value can be validly duplicated by copying its bits, then calling `.clone()` on it can simply perform that same bitwise copy, and often does exactly that in generated code. But the reverse does not hold. Many `Clone` types, `String` chief among them, are not and cannot be `Copy`, because their duplication is not a bit-level operation at all. `Copy` is implicit: it happens on ordinary assignment without any method call, and the language only grants it where implicit duplication is always safe. `Clone` is explicit: it requires the programmer to write `.clone()`, precisely because the operation it performs may not be safe, or cheap, to run silently on every assignment.

7. Deriving the traits

Both traits can usually be derived rather than implemented by hand.

```
#[derive(Copy, Clone)]
struct Point {
    x: i32,
    y: i32,
}
```

`Point` may derive `Copy` because every field it contains is itself `Copy`; duplicating the whole struct bitwise is exactly as safe as duplicating each of its fields. Contrast this with a struct containing a `String`:

```
#[derive(Clone)]
struct Ticket {
    label: String,
}
```

`Ticket` may derive `Clone`, since cloning it just means cloning its `String` field. It cannot derive `Copy`: the compiler will refuse, because one of its fields is not `Copy`, and there is no way to bitwise-duplicate a `Ticket` without also bitwise-duplicating a heap handle that must not be duplicated.

8. Duplication cost and semantic cost

Writing `.clone()` keeps the choice to reconstruct a value visible in the source text, but visibility is not the same as cheapness. Cloning an empty `Vec` is nearly free; cloning a ten-million-element `Vec` is not. The syntax `.clone()` means only “perform this type’s defined reconstruction procedure,” and what that procedure actually costs depends entirely on what the type is. The value of the explicit call is not that it guarantees cheapness, but that it guarantees the decision to reconstruct was made deliberately, at a specific, readable point in the program, rather than happening invisibly on every assignment.

9. `Rc::clone` does not clone the underlying value

Chapter 5 introduced `Rc<T>` as shared, reference-counted ownership. Calling `.clone()` on an `Rc<T>` is instructive precisely because it looks like the `String` case above but means something different underneath.

```
use std::rc::Rc;

let a = Rc::new(String::from("shared"));
let b = Rc::clone(&a);
```

This does not allocate a second `String`. It increments a reference count and produces a second handle that points at the same allocation as the first. Both `a` and `b` now participate in the same underlying object; neither has an independent future the way the cloned `String` in Section 5 did. This is not

a contradiction of what `Clone` means — it is a demonstration that `Clone` names whatever reconstruction procedure a type defines for itself, and that procedure need not produce an independent value at all. `Rc`'s procedure is defined to produce a new, equally authoritative reference to a shared value, not a second value.

10. Copying state versus preserving identity

Three operations, three different answers to what survives duplication. A copied `i32` is fully interchangeable with its source; there was never a meaningful sense in which the two “shared a history” to begin with, since integers carry no history beyond their value. A cloned `String` begins with equal contents but immediately diverges into an independent future; it has its source's present state and none of its subsequent one. A cloned `Rc` handle shares not just contents but an ongoing, single future with its source — the two names are, and remain, two ways of reaching one continuing object. `Move`, `Copy`, and `Clone` are not three implementations of one idea called “duplication.” They are three different, precise answers to the question of what is permitted to continue after an assignment: authority may transfer entire and undivided, an interchangeable value may be freely duplicated, or a new and independent continuation may be explicitly reconstructed on request.

Invariant Established

`Move`, `Copy`, and `Clone` answer the same question — what may continue after assignment — with three different resolutions: transferred authority, interchangeable duplication, or explicitly reconstructed independence. The operation chosen for a given type determines which distinctions — identity, history, shared future — survive the appearance of a second name.

Non-overlap note: Chapter 5 establishes uniqueness of authority and explains movement in general. This interlude teaches the language-level vocabulary — `Copy`, `Clone`, and their derivation — that Chapter 5 assumes but does not separately develop.